

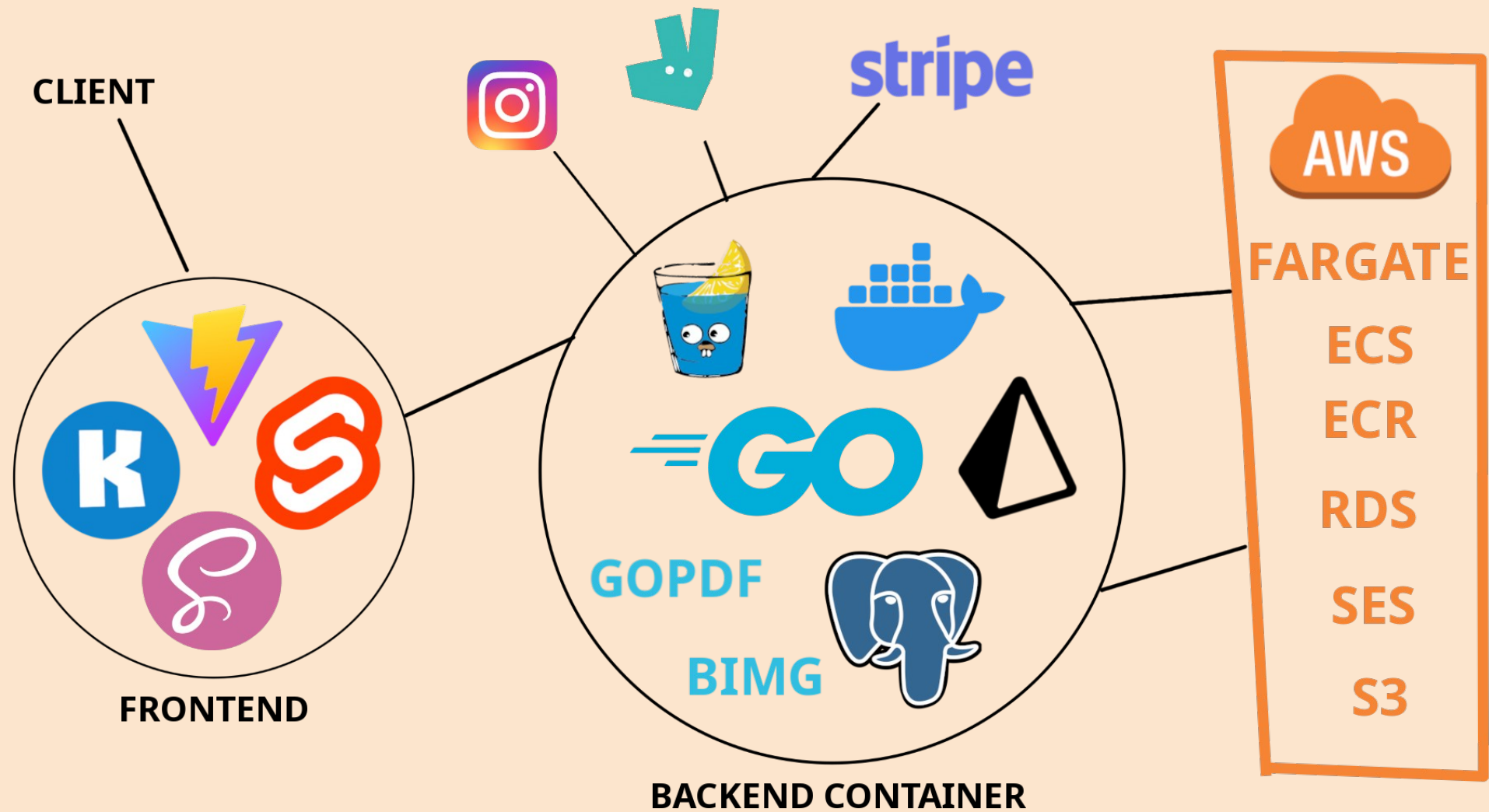
=-

Spécifications Techniques

[Menu Maker by Qwenta]

Version	Auteur	Date	Approbation
1.0	Toby Foster	20/05/2025	Florent Houx

I. Choix technologiques	2
II. Liens avec le back-end	3
III. Préconisations concernant le domaine et l'hébergement	3
IV. Accessibilité	3
V. Recommandations en termes de sécurité	3
VI. Maintenance du site et futures mises à jour	4



I. Choix technologiques

- État des lieux des besoins fonctionnels et de leurs solutions techniques :

Besoin	Contraintes	Solution	Description de la solution	Justification (2 arguments)
Création d'une catégorie de menu	L'ajout d'une catégorie doit pouvoir se faire directement sur l'écran de création de menu depuis une modale.	Svelte-modal	Une fenêtre contextuelle basique JavaScript créée avec Svelte. Utilisez-la comme composant de base pour créer des fenêtres contextuelles agréables et utiles, comme svelte-dialog.	1) Nous avons choisi de développer en Svelte, la librairie est cohérente avec ce choix. 2) Il s'agit de la librairie la plus utilisée.
Un serveur	Prix (bare metal, machine virtuelle, conteneur, serverless)	Docker	Docker permet d'exécuter des applications dans des conteneurs légers, portables et isolés. C'est une solution idéale pour le déploiement rapide, la scalabilité et l'efficacité des ressources. Il est particulièrement adapté aux architectures modernes comme les microservices et les applications web.	1) Scalabilité : Docker facilite la montée en charge horizontale. On peut lancer plusieurs instances du conteneur pour gérer une charge accrue, et utiliser des orchestrateurs comme Kubernetes pour automatiser cette gestion. 2) Portabilité et cohérence : Docker regroupe l'application et toutes ses dépendances dans un conteneur unique, ce qui garantit un fonctionnement identique en développement, en test ou en production. Cela élimine les problèmes liés aux différences d'environnement.
Connexion par e-mail	Envoyer un e-mail depuis le backend, contenant un	Créer un système	Quand un utilisateur entre son e-mail, le backend envoie un lien temporaire	1) Go est plus adapté pour les microservices : il est léger, rapide, et

(login sans mot de passe)	lien avec un token unique que l'utilisateur doit cliquer pour se connecter.	d'authentification par lien magique (magic link) via une API backend en Go.	avec un token sécurisé. Quand il clique dessus, le serveur vérifie le token, et s'il est valide, il connecte l'utilisateur.	fonctionne très bien dans des conteneurs Docker, surtout avec des outils comme AWS Fargate. 2) Plus simple à maintenir qu'un système complet d'authentification juste des tokens temporaires faciles à créer et vérifier.
Créer un tableau de bord après connexion.	Navigation avec onglets sur le côté Accès rapide aux fonctionnalités clés	SvelteKit + Routage dynamique	Interface avec barre latérale fixe et accès rapide aux fonctions principales (créer, diffuser, imprimer un menu). Utiliser SvelteKit avec un système d'onglets basé sur le routage pour structurer le tableau de bord.	1) Composants modulables et réactifs 2) Navigation rapide avec chargement partiel
Créer, personnaliser, exporter et diffuser ses menus.	Interface intuitive pour l'ajout/modif Aperçu visuel du menu Export PDF, diffusion web	SvelteKit + Konva.js	Konva.js est une bibliothèque JavaScript pour créer et manipuler des formes 2D sur le canevas HTML5. Elle permet une gestion fine des objets (textes, images, formes) et offre des fonctionnalités de zoom, de déplacement, ainsi qu'un système de grille et de magnétisme pour une expérience d'édition précise et permet d'éditer un menu interactif sur canvas avec custom fonts/images.	1) Plus puissant que Fabric.js et Paper.js : Konva.js offre des performances optimisées et un contrôle plus précis sur la manipulation des éléments, ce qui en fait une option plus robuste pour des applications de création graphique complexes. 2) Gestion manuelle de l'état : Contrairement à des bibliothèques de niveau supérieur, Konva permet une gestion manuelle de l'état de chaque élément (position, taille, rotation), ce qui donne plus de flexibilité pour personnaliser et adapter l'éditeur aux besoins spécifiques du projet.

Modifier les infos du restaurant.	Interface sécurisée Champs de formulaire clairs Sauvegarde en base PostgreSQL	SvelteKit + API Go	Utilisation de formulaires liés à une API Go qui enregistre les données dans PostgreSQL.	1) SvelteKit pour interface rapide 2) Go + PostgreSQL = performance
-----------------------------------	---	--------------------	--	--

II. Liens avec le back-end

- Quel langage pour le serveur ?

Pour le serveur, j'ai utilisé Go (Golang). C'est plus simple et plus rapide que Node.js et Python dans un conteneur Docker. Go utilise moins de ressources, il démarre plus vite, et il peut faire plusieurs choses en même temps grâce à sa concurrence native, un peu comme async, mais intégré dans le langage avec ce qu'on appelle les goroutines. C'est aussi mieux pour la scalabilité. En plus, Go marche très bien avec AWS, surtout avec Fargate, donc c'est plus facile à déployer et à faire évoluer.

- A-t-on besoin d'une API ? Si oui laquelle ?

Oui, j'ai besoin d'une API pour connecter tous les services que j'utilise, notamment AWS RDS pour PostgreSQL, AWS S3 pour le stockage d'images et de PDF, ainsi que Stripe pour les paiements. Je peux créer une API REST en utilisant Go avec le framework Gin. Elle permettra de :

- Gérer les opérations CRUD sur PostgreSQL via des endpoints.
- Interagir avec AWS S3 pour télécharger et récupérer des fichiers (images, PDF).
- Connecter Stripe pour le traitement des paiements.

En ce qui concerne l'authentification par e-mail, l'API en Go permettra de générer un lien magique (magic link) pour les utilisateurs.

Lorsqu'un utilisateur entre son e-mail, le backend génère un token sécurisé et envoie un e-mail via net/smtp ou SES pour les services de messagerie. L'e-mail contiendra un lien que l'utilisateur pourra cliquer pour se connecter à l'application.

Si Qwenta possède déjà un serveur de messagerie, je peux directement utiliser net/smtp pour envoyer les e-mails, mais si besoin, j'opterais pour SES pour une meilleure gestion et fiabilité des envois.

- Base de données choisie : *Ex : SQL / NO SQL.*

J'ai choisi PostgreSQL comme base de données, car il prend en charge l'évolutivité horizontale, ce qui est essentiel pour une application qui sera déployée sur plusieurs serveurs à travers le monde. Il peut gérer plusieurs transactions simultanément et offre une meilleure durabilité par rapport à une base de données NoSQL comme MongoDB. De plus, il ne nécessite pas de gérer d'énormes quantités de données, ce qui le rend idéal pour mon projet. Il combine donc les avantages des bases de données relationnelles et NoSQL.

III. Préconisations concernant le domaine et l'hébergement

- Nom de domaine :

Créer un sous-domaine, le client a toutes les possibilités qu'il souhaite tant qu'il n'est pas déjà utilisé.

1. **menu-maker.qwenta.ch**
2. **menumaker.qwenta.ch**
3. **menu-maker-by.qwenta.ch**

- L'hébergement : **AWS**

- **ECR (Elastic Container Registry)** me permet de stocker les images Docker de manière sécurisée dans le cloud.
- **ECS (Elastic Container Service)** me permet de déployer et gérer ces conteneurs facilement, en assurant l'orchestration et la scalabilité de l'application.

- **Fargate** est utilisé avec ECS pour exécuter les conteneurs sans gérer de serveurs, ce qui simplifie le déploiement, réduit les coûts et permet une meilleure intégration avec le reste des services AWS.
- **RDS (Relational Database Service)** héberge la base de données PostgreSQL avec une gestion simplifiée (sauvegardes, mises à jour, haute disponibilité), tout en étant performant et compatible avec les besoins de l'app.
- **S3 (Simple Storage Service)** est utilisé pour stocker les fichiers comme les images et les PDF, avec une haute disponibilité, un accès rapide, et une intégration facile avec le reste du stack.
- **SES (Simple Email Service)** Authentification via lien magique (email sécurisé)

- Adresses e-mail :

Le site de qwenta.ch ne présente pas d'adresses e-mail publiques, seulement un formulaire de contact. Je suppose donc qu'ils souhaiteraient adopter la même approche pour ce projet. Bien qu'aucun formulaire de contact ne soit prévu dans la maquette Figma, il serait simple d'en ajouter un s'ils le souhaitent. Par ailleurs, possédant leur propre nom de domaine, ils peuvent créer n'importe quelle adresse e-mail selon leurs besoins.

IV. Accessibilité

- Compatibilité navigateur.

Le support des navigateurs pour le site devrait se concentrer sur Chrome, Firefox, Safari, et Edge, en excluant Internet Explorer, qui est de moins en moins utilisé.

- Types d'appareils.

Pour commencer, le site sera en mode bureau uniquement, car développer un générateur de menu pour mobile avec uniquement des commandes tactiles représenterait un défi pour l'expérience utilisateur.

V. Recommandations en termes de sécurité

- Accès aux comptes, plugins...
 1. Authentification :

On utilise un système de connexion par lien magique. Quand l'utilisateur entre son e-mail, le backend en Go avec Gin : Quand l'utilisateur clique sur le lien, l'API vérifie si le token est valide (existe + pas expiré) et le connecte. Pas besoin de mot de passe = moins de failles et plus simple à maintenir. Pour envoyer les e-mails, on peut utiliser net/smtp ou un service comme SES.
 2. Qwenta a déjà un système de connexion. On pourra voir avec eux s'ils veulent fusionner les comptes pour que leurs clients puissent utiliser ce service directement, avec une seule connexion. Ça rendra l'expérience plus fluide pour les utilisateurs.

VI. Maintenance du site et futures mises à jour

- Grandes lignes du contrat de maintenance.
 1. **Surveillance et performance**
 - Suivi des performances (temps de réponse, erreurs serveur).
 - Outils de monitoring mis à jour régulièrement.
 2. **Mises à jour de sécurité**

- Mises à jour régulières des dépendances (Go, bibliothèques, outils Docker).
- Application rapide des correctifs de sécurité.

3. Sauvegardes

- Sauvegardes automatiques de la base PostgreSQL (via RDS) et des fichiers sur S3.
- Tests réguliers de restauration.

4. Évolutions du site

- Ajout de nouvelles fonctionnalités selon les retours utilisateurs.
- Intégration possible avec des outils comme le système de login de Qwenta ou de nouveaux moyens de paiement.

5. Corrections de bugs

- Résolution rapide des erreurs signalées.
- Suivi des problèmes avec un système de tickets.

6. Tests de compatibilité

- Vérification régulière sur les navigateurs récents (desktop seulement au début).
- Tests de performance pour s'adapter à l'évolution du trafic.

7. Optimisation

- Optimisation du code Go, des requêtes SQL, et des ressources statiques.
- Révision de l'architecture si le site évolue fortement.

8. Assistance technique

- Support technique disponible pour les urgences ou questions.
- Délai de réponse garanti selon le type de demande.

Résumé

Le contrat de maintenance couvre la sécurité, les performances, les sauvegardes, les corrections de bugs, l'évolution continue du site et une assistance technique fiable pour assurer le bon fonctionnement du service dans le temps.